LARAVEL 12
COMPLETE REVISION & TEACHING MANUAL
Lessons 1–5 | Concepts • Commands • Code • Practical Exercises

CLASSROOM EDITION • LARAVEL 12

Purpose: A classroom-ready guide for revising the Laravel syllabus shown in the supplied lesson plan. Examples use Laravel 12 and PHP 8.2+ conventions. Some commands may differ slightly depending on the installed PHP, Composer, database driver, and Laravel patch version.

Learning outcomes

- Explain what Laravel is and why developers use it instead of writing every feature from scratch in traditional PHP.
- Install the development environment and create a Laravel project with Composer.
- Identify the important directories and files in a Laravel application.
- Explain MVC and trace a browser request through routes, controllers, models and views.
- Create GET and POST routes, use route parameters, and generate named-route URLs.
- Create Blade views, pass data, use directives, layouts, sections and includes.
- Create controllers, connect them to routes and views, and use resource routes.
- Understand middleware and apply it to selected routes or controller actions.

How the five lessons connect

We will build a small Student Management example. A browser requests /students. A route receives the request and calls a controller. The controller prepares data and returns a Blade view. A form sends a POST request to store a student. Middleware can check a request before it reaches the controller. In a database-backed application, a Model would retrieve or save records; this revision example uses an in-memory array first so the routing and MVC flow are easy to see.

Lesson 1 — Introduction to Laravel

1.1 What is Laravel?

Laravel is an open-source PHP web application framework. It provides a structured way to build web applications and APIs. Instead of manually writing common features repeatedly, developers use Laravel's routing, controllers, Blade templating, middleware, validation, database tools, authentication ecosystem, queues and command-line tools.

1.2 Features and advantages

Feature	Meaning / classroom explanation
Routing	Maps a URL and HTTP method to a closure or controller action.
MVC structure	Separates request handling, application logic/data access and presentation.
Blade	A server-side template engine for reusable HTML views.
Middleware	Filters or inspects HTTP requests before/after route handling.
Artisan	Command-line interface for generating files and running framework tasks.
Eloquent ORM	Laravel's object-relational mapper for working with database records.
Migrations	Version-controlled PHP files that define database schema changes.
Validation	Convenient rules for checking incoming form/API data.
Security helpers	Includes CSRF protection, password hashing and query parameter binding.
Ecosystem	Official packages and tools support authentication, queues, testing and deployment.

1.3 Laravel compared with traditional PHP

Traditional PHP approach	Laravel approach
Developer manually maps URLs or mixes PHP into pages.	Routes are centrally defined in route files.
Repeated include files and custom conventions.	A consistent project structure and dependency container.
HTML and PHP logic often become mixed together.	Blade separates presentation into templates.
Forms need manually built CSRF checks and validation.	Built-in CSRF middleware and validation facilities.
Database queries and connection handling are often handwritten.	Database configuration, query builder and Eloquent are available.
Reusable patterns depend on each developer.	Controllers, middleware, migrations and other conventions encourage reuse.

Important: Traditional PHP is not inherently bad or insecure. PHP can be well-structured without Laravel. Laravel mainly provides conventions and ready-made tools, reducing repetitive work and making larger applications easier to maintain.

1.4 What is Composer?

Composer is PHP's dependency manager. It installs libraries, resolves compatible package versions and generates an autoloader. Laravel projects use Composer to install framework dependencies and manage packages.

```
composer --version
php --version
composer create-project laravel/laravel student-portal
cd student-portal
php artisan --version
php artisan serve
```

Open the local address printed by Artisan (commonly http://127.0.0.1:8000). Keep the development server terminal running while testing.

1.5 Development environment checklist

- PHP version supported by the selected Laravel 12 release, with required PHP extensions enabled.
- Composer installed and available in the terminal PATH.
- Code editor such as VS Code.
- Database server only if the application needs persistent database storage.
- Node.js and npm if compiling frontend assets with Vite.
- Browser and API client such as Thunder Client or Postman for testing.

1.6 Environment configuration

The .env file stores environment-specific configuration such as application name, environment, URL and database credentials. Do not commit real secrets or production credentials to a public repository. Laravel reads configuration through config files, which commonly reference env values.

```
APP_NAME="Student Portal"
APP_ENV=local
APP_DEBUG=true
APP_URL=http://127.0.0.1:8000

# Example database settings (use your own credentials)
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=student_portal
DB_USERNAME=root
DB_PASSWORD=
```

Never use APP_DEBUG=true on a live production server. Debug pages can expose sensitive application details.

Lesson 2 — Create a Laravel project, directory structure and Artisan

2.1 Create and run a project

```
composer create-project laravel/laravel student-portal
cd student-portal
php artisan serve
```

Alternative: if the Laravel installer is installed, you can use laravel new student-portal. The Composer create-project command is sufficient for this lesson.

2.2 Important directory and file structure

Path	Purpose
app/	Core application code: models, providers, and application classes.
app/Http/Controllers/	Controller classes that handle requests and coordinate responses.
app/Http/Middleware/	Custom middleware classes.
app/Models/	Eloquent model classes.
bootstrap/	Framework bootstrapping and application configuration.
config/	Application and package configuration files.
database/migrations/	Database schema change files.
database/seeders/	Classes that insert sample or initial database data.
public/	Public entry point (index.php) and publicly served assets.
resources/views/	Blade view templates.
resources/css/ and resources/js/	Frontend source assets, commonly built with Vite.
routes/web.php	Routes for browser-based web pages; web middleware includes session and CSRF handling.
routes/api.php	API routes when API routing is installed/configured in the project.
storage/	Logs, cache, sessions and generated application files.
tests/	Automated tests.
vendor/	Composer-installed dependencies; do not edit framework vendor files.
.env	Local environment values and secrets; keep private.
composer.json	PHP package requirements and Composer scripts.
artisan	Command-line entry point for Laravel Artisan.

2.3 What is Artisan?

Artisan is Laravel's command-line interface. It helps create classes, inspect routes, run migrations, clear caches and perform many development tasks.

Command	Use
php artisan list	List available commands.
php artisan --version	Display Laravel version.
php artisan about	Show information about the application.
php artisan serve	Start the local development server.
php artisan route:list	List registered routes.
php artisan make:controller StudentController	Generate a controller.
php artisan make:controller StudentController --resource	Generate a resource controller.
php artisan make:model Student	Generate a model.
php artisan make:middleware CheckAge	Generate middleware.
php artisan make:view students.index	Generate a view (supported in current Laravel versions).
php artisan migrate	Run pending migrations.
php artisan optimize:clear	Clear common cached/optimized files.

2.4 Explore the project

```
# Show all registered routes
php artisan route:list

# Generate a controller
php artisan make:controller StudentController

# Generate a resource controller
php artisan make:controller StudentController --resource

# Generate custom middleware
php artisan make:middleware CheckAge
```

Do not randomly delete files to make the project smaller. Understand which files are generated, which are configuration, and which are runtime dependencies.

Lesson 3 — MVC architecture and routing

3.1 Understand MVC

Part	Responsibility	Example in this project
Model	Represents data and database interaction.	Student model (when database storage is added).
View	Displays the user interface.	resources/views/students/index.blade.php
Controller	Handles application flow and prepares a response.	StudentController@index

Typical request flow: Browser → routes/web.php → middleware → controller action → model/database if needed → Blade view or another response → Browser. A route may return a view directly for a very small page, but substantial application logic belongs in controllers/services rather than being placed in route closures.

3.2 What is a route?

A route connects an HTTP method and URI to code that handles the request. Web routes are commonly defined in routes/web.php.

```
use Illuminate\\Support\\Facades\\Route;

Route::get('/hello', function () {
    return 'Hello Laravel';
});
```

Visit /hello in the browser. The route returns a plain-text response.

3.3 GET and POST routes

GET is normally used to retrieve a page or resource. POST is normally used to submit data or create a resource. The HTTP method is part of the route definition; a GET route does not automatically handle POST.

```
use Illuminate\\Support\\Facades\\Route;

Route::get('/contact', function () {
    return view('contact');
});

Route::post('/contact', function (\\Illuminate\\Http\\Request $request) {
    $name = $request->input('name');
```

```
    return 'Thank you, ' . $name;
});
```

3.4 Create a POST form with CSRF protection

Create resources/views/contact.blade.php. A form submitted to a Laravel web route must include @csrf. Laravel's web middleware checks the CSRF token to help prevent forged form submissions.

```
<form method="POST" action="{{ url('/contact') }}">
    @csrf
    <label>Name</label>
    <input type="text" name="name" required>
    <button type="submit">Send</button>
</form>
```

If a POST request made through Thunder Client returns HTTP 419 Page Expired, the request likely lacks a valid CSRF token or session. For a web route, first load the form/session and send the token and session cookie. Do not disable CSRF protection just to make the test pass. API routes use a different middleware setup and should be designed deliberately.

3.5 Route parameters

A required route parameter is written inside curly braces. It becomes an argument to the route closure or controller method.

```
// Required parameter
Route::get('/students/{id}', function (string $id) {
    return 'Student ID: ' . $id;
});

// Multiple parameters
Route::get('/courses/{course}/students/{student}', function ($course, $student) {
    return "Course: $course, Student: $student";
});

// Optional parameter (must have a default value)
Route::get('/welcome/{name?}', function (?string $name = 'Guest') {
    return 'Welcome ' . $name;
});
```

3.6 Route constraints

```
Route::get('/students/{id}', function (string $id) {
    return "Student $id";
})->whereNumber('id');

Route::get('/posts/{slug}', function (string $slug) {
    return $slug;
})->where('slug', '[A-Za-z0-9-]+');
```

3.7 Named routes

A named route has a stable identifier. Use route('name') to generate its URL instead of hardcoding the path throughout the application. If the URI changes later, code using the route name can continue working.

```
Route::get('/students', function () {
    return 'All students';
})->name('students.index');

// Generate URL in PHP:
$url = route('students.index');
```

```
// Generate URL with a parameter:
Route::get('/students/{id}', function ($id) {
    return "Student $id";
})->name('students.show');

$url = route('students.show', ['id' => 5]);
```

In Blade: All Students

3.8 Route order and route:list

Put specific routes before broad wildcard routes when their patterns could conflict. Use php artisan route:list to inspect methods, URIs, names, actions and middleware. Route names should be unique.

Lesson 4 — Views, Blade and template inheritance

4.1 What is a view?

A view is the presentation layer. Laravel Blade files usually end with .blade.php and are stored in resources/views. A view should primarily display data, not perform database queries or contain complex business logic.

4.2 Return a view from a route

```
Route::get('/about', function () {
    return view('about');
});
```

This loads resources/views/about.blade.php. Dot notation maps folders to paths: view('students.profile') loads resources/views/students/profile.blade.php.

4.3 Passing data to a view

```
Route::get('/student-profile', function () {
    $student = 'Ayesha';
    $course = 'Laravel';

    return view('student-profile', [
        'studentName' => $student,
        'courseName' => $course,
    ]);
});
```

In student-profile.blade.php:

```
<h1>Student: {{ $studentName }}</h1>
<p>Course: {{ $courseName }}</p>
```

You can also use compact('student', 'course') when the variable names and view keys should match.

4.4 Blade output and escaping

```
{{ $name }}       {{-- Escaped output: use this for normal user data --}}
{!! $html !!}     {{-- Unescaped HTML: use only for trusted/sanitized HTML --}}
```

Use {{ }} for ordinary output. Printing untrusted input with {!! !!} can create cross-site scripting (XSS) vulnerabilities.

4.5 Common Blade directives

Directive	Purpose / example
@if / @elseif / @else / @endif	Conditional output.

@unless / @endunless	Render unless a condition is true.
@isset / @endisset	Check whether a variable is set.
@empty / @endempty	Check whether a value is empty.
@foreach / @endforeach	Loop through an array or collection.
@forelse / @empty / @endforelse	Loop with an empty-state branch.
@for / @endfor	Traditional counting loop.
@while / @endwhile	While loop.
@csrf	Insert hidden CSRF token field.
@method('PUT')	Spoof an HTTP method in an HTML form.
@include('partials.nav')	Include another Blade file.
@extends('layouts.app')	Inherit a parent layout.
@section / @endsection	Define content for a layout section.
@yield('content')	Display a section defined by a child view.
@push / @endpush and @stack	Add and render stacks such as scripts.

4.6 Conditional example

```
@if ($marks >= 40)
    <p>Result: Pass</p>
@else
    <p>Result: Fail</p>
@endif
```

4.7 Loop example

```
@php
    $students = ['Ayesha', 'Rahul', 'Sana'];
@endphp

<ul>
    @foreach ($students as $student)
        <li>{{ $loop->iteration }}. {{ $student }}</li>
    @endforeach
</ul>
```

$loop->iteration starts at 1. Other useful properties include $loop->index (starts at 0), $loop->first, $loop->last and $loop->count.

4.8 @forelse empty-state example

```
@forelse ($students as $student)
    <p>{{ $student }}</p>
@empty
    <p>No students found.</p>
@endforelse
```

4.9 Layouts and template inheritance

A layout stores the common HTML structure (head, navigation, footer). Child views extend it and provide section content. This avoids repeating the same HTML on every page.

```
<!-- resources/views/layouts/app.blade.php -->
<!DOCTYPE html>
<html>
<head>
    <title>@yield('title', 'Student Portal')</title>
</head>
<body>
    @include('partials.nav')

    <main>
        @yield('content')
    </main>
```

```
    @include('partials.footer')
</body>
</html>
```

```
<!-- resources/views/students/index.blade.php -->
@extends('layouts.app')

@section('title', 'Students')

@section('content')
    <h1>Student List</h1>
    <p>Welcome to the student portal.</p>
@endsection
```

4.10 Including reusable views

```
<!-- resources/views/partials/nav.blade.php -->
<nav>
    <a href="{{ route('students.index') }}">Students</a>
    <a href="{{ route('students.create') }}">Add Student</a>
</nav>

<!-- Include from another view -->
@include('partials.nav')
```

Use @include for reusable partial markup. Use @extends/@section/@yield for a parent-child page layout. The included view can receive data explicitly, for example @include('partials.card', ['title' => 'Notice']).

Lesson 5 — Controllers, resource controllers and middleware

5.1 What is a controller?

A controller is a PHP class that groups related request-handling methods. It receives route parameters and request data, coordinates application work, and returns a response such as a view, redirect or JSON response.

5.2 Create a controller

```
php artisan make:controller StudentController
```

This creates app/Http/Controllers/StudentController.php.

```
<?php

namespace App\\Http\\Controllers;

use Illuminate\\Http\\Request;

class StudentController extends Controller
{
    public function index()
    {
        $students = ['Ayesha', 'Rahul', 'Sana'];
        return view('students.index', compact('students'));
    }

    public function show(string $id)
    {
        return "Student ID: " . $id;
```

```
    }
}
```

5.3 Connect routes to controller methods

```
use App\\Http\\Controllers\\StudentController;
use Illuminate\\Support\\Facades\\Route;

Route::get('/students', [StudentController::class, 'index'])
    ->name('students.index');

Route::get('/students/{id}', [StudentController::class, 'show'])
    ->whereNumber('id')
    ->name('students.show');
```

The array [StudentController::class, 'index'] tells Laravel which class and method should handle the request. The route name is separate from the controller method name.

5.4 Build a small complete GET + POST example

For teaching request flow without a database, use a static array. Data added during a POST request will not persist after the request; this is intentional for a routing demonstration. Use a database and Model for persistent storage.

```
// routes/web.php
use App\\Http\\Controllers\\StudentController;
use Illuminate\\Support\\Facades\\Route;

Route::get('/students', [StudentController::class, 'index'])
    ->name('students.index');
Route::get('/students/create', [StudentController::class, 'create'])
    ->name('students.create');
Route::post('/students', [StudentController::class, 'store'])
    ->name('students.store');
```

```
// app/Http/Controllers/StudentController.php
namespace App\\Http\\Controllers;

use Illuminate\\Http\\Request;

class StudentController extends Controller
{
    public function index()
    {
        $students = ['Ayesha', 'Rahul', 'Sana'];
        return view('students.index', compact('students'));
    }

    public function create()
    {
        return view('students.create');
    }

    public function store(Request $request)
    {
        $validated = $request->validate([
            'name' => ['required', 'string', 'max:100'],
            'email' => ['required', 'email'],
        ]);

        // Demonstration only: validated data is not saved permanently.
        return redirect()->route('students.index')
            ->with('success', 'Student details received for ' . $validated['name']);
```

```
    }
}
```

```
<!-- resources/views/students/index.blade.php -->
@extends('layouts.app')
@section('title', 'Students')
@section('content')
    <h1>Students</h1>
    @if (session('success'))
        <p>{{ session('success') }}</p>
    @endif
    <a href="{{ route('students.create') }}">Add student</a>
    <ul>
        @foreach ($students as $student)
            <li>{{ $student }}</li>
        @endforeach
    </ul>
@endsection
```

```
<!-- resources/views/students/create.blade.php -->
@extends('layouts.app')
@section('title', 'Add Student')
@section('content')
    <h1>Add Student</h1>
    <form method="POST" action="{{ route('students.store') }}">
        @csrf
        <label>Name</label>
        <input name="name" value="{{ old('name') }}">
        @error('name') <p>{{ $message }}</p> @enderror

        <label>Email</label>
        <input type="email" name="email" value="{{ old('email') }}">
        @error('email') <p>{{ $message }}</p> @enderror

        <button type="submit">Save</button>
    </form>
@endsection
```

5.5 Resource controllers

A resource controller follows conventional CRUD actions: index, create, store, show, edit, update and destroy. It saves time by mapping a resource route to these standard controller methods.

```
php artisan make:controller StudentController --resource
```

```
use App\\Http\\Controllers\\StudentController;
use Illuminate\\Support\\Facades\\Route;

Route::resource('students', StudentController::class);
```

HTTP method	URI	Controller method	Typical purpose
GET	/students	index	List all students
GET	/students/create	create	Show create form
POST	/students	store	Save a new student
GET	/students/{student}	show	Show one student
GET	/students/{student}/edit	edit	Show edit form
PUT/PATCH	/students/{student}	update	Update a student
DELETE	/students/{student}	destroy	Delete a student

Use php artisan route:list to confirm the exact generated routes. You can limit a resource controller to selected actions: Route::resource('students', StudentController::class)->only(['index', 'show']); or exclude actions with ->except([...]).

5.6 HTML forms and PUT/DELETE

HTML forms natively support GET and POST. Laravel uses a hidden _method field to simulate PUT, PATCH or DELETE. Blade provides @method.

```
<form method="POST" action="{{ route('students.update', $student->id) }}">
    @csrf
    @method('PUT')
    <input name="name" value="{{ old('name', $student->name) }}">
    <button type="submit">Update</button>
</form>

<form method="POST" action="{{ route('students.destroy', $student->id) }}">
    @csrf
    @method('DELETE')
    <button type="submit">Delete</button>
</form>
```

5.7 What is middleware?

Middleware is a request filter. It can inspect a request, reject it, redirect it, or allow it to continue. Examples include authentication checks, role checks, request logging and age restrictions. Middleware is not the same as validation: validation checks submitted data, while middleware controls whether a request should proceed.

5.8 Create custom middleware

```
php artisan make:middleware CheckAge
```

```
// app/Http/Middleware/CheckAge.php
namespace App\\Http\\Middleware;

use Closure;
use Illuminate\\Http\\Request;
use Symfony\\Component\\HttpFoundation\\Response;

class CheckAge
{
    public function handle(Request $request, Closure $next): Response
    {
        if ((int) $request->input('age', 0) < 18) {
            return response('You must be at least 18 years old.', 403);
        }

        return $next($request);
    }
}
```

$next($request) passes the request to the next middleware or route. If middleware returns a response before calling $next, the controller is not reached.

5.9 Register and apply middleware in Laravel 12

In current Laravel 12 application structure, middleware aliases and groups are commonly configured in bootstrap/app.php. The exact file may differ in upgraded or customized projects. Add an alias inside the withMiddleware callback:

```
// bootstrap/app.php (illustrative placement inside ->withMiddleware(...))
use Illuminate\\Foundation\\Configuration\\Middleware;

->withMiddleware(function (Middleware $middleware): void {
    $middleware->alias([
        'checkage' => \\App\\Http\\Middleware\\CheckAge::class,
```

```
    ]);
})
```

Then apply it to a route:

```
Route::get('/restricted', function () {
    return 'Access granted';
})->middleware('checkage');
```

Or apply it to a group:

```
Route::middleware(['checkage'])->group(function () {
    Route::get('/restricted-one', fn () => 'One');
    Route::get('/restricted-two', fn () => 'Two');
});
```

5.10 Controller middleware in Laravel 12

Laravel 12 supports controller middleware through the HasMiddleware contract in versions that provide this controller feature. This is a static method returning middleware definitions. The array is a PHP list of middleware objects; only is an argument that limits the middleware to named controller actions.

```
namespace App\\Http\\Controllers;

use Illuminate\\Routing\\Controllers\\HasMiddleware;
use Illuminate\\Routing\\Controllers\\Middleware;

class StudentController extends Controller implements HasMiddleware
{
    public static function middleware(): array
    {
        return [
            new Middleware('checkage', only: ['create', 'store']),
        ];
    }

    public function index() { /* ... */ }
    public function create() { /* ... */ }
    public function store() { /* ... */ }
}
```

If your installed Laravel 12 patch or project structure does not expose this contract, apply middleware directly to routes or route groups. Route middleware is explicit and is often easier for beginners to debug.

5.11 Middleware execution flow

- Client sends a request.
- Global middleware runs, followed by applicable route middleware.
- Middleware may return a response (request blocked/redirected), or call $next($request).
- After middleware allows the request, Laravel executes the route/controller.
- The response travels back through middleware that performs response-side work, then to the client.

Key concept comparisons

Concepts	Difference
Route vs Controller	A route maps an HTTP method/URI; a controller method contains the request-handling logic.
Controller vs Model	A controller coordinates a request; a model represents data and database interaction.
View vs Blade	A view is the presentation file; Blade is Laravel's template engine used by .blade.php files.

GET vs POST	GET retrieves; POST submits/processes data or creates a resource.
Named route vs URL string	A route name is a stable identifier; a URL string is a hardcoded path.
Middleware vs validation	Middleware controls request access/flow; validation checks data correctness.
@include vs @extends	@include inserts a partial; @extends inherits a parent layout.
Resource route vs one route	A resource route generates conventional CRUD routes; a single route maps one method/URI.

Common errors and troubleshooting

Error / symptom	Likely cause and fix
404 Not Found	Check URI, HTTP method, route registration and route:list.
405 Method Not Allowed	The URL exists but not for the HTTP method used. Check GET/POST/PUT/DELETE.
419 Page Expired	A web POST/PUT/PATCH/DELETE request is missing a valid CSRF token/session. Include @csrf and preserve cookies.
Target class does not exist	Check namespace, import, class/file name, spelling and Composer autoload.
View not found	Check resources/views path and dot notation; filename must end in .blade.php.
Undefined variable	Pass the variable from the route/controller to the view using an array or compact().
Route [name] not defined	Check ->name(...), spelling and route:list.
Changes not appearing	Check correct project/server, route cache and view cache; use php artisan optimize:clear during local troubleshooting.
Class not found after creating a class	Check namespace and file path; run composer dump-autoload if appropriate.

Practical classroom exercise

Task: Build a basic Student Portal

- Create a Laravel project named student-portal and run it.
- Create a /welcome GET route that returns a Blade view.
- Create a named route students.index for /students.
- Create a StudentController with index(), create() and store() methods.
- Display a student list in a Blade view using @foreach and $loop->iteration.
- Create a student form with name and email fields, @csrf and validation error output.
- Create a POST route that calls store(). Validate name and email, then redirect back with a success flash message.
- Create a custom CheckAge middleware and protect a sample route.
- Register middleware alias in bootstrap/app.php and test allowed/blocked requests.
- Run php artisan route:list and explain each route.

Extension exercise

Convert the in-memory student list to database storage. Create a Student model and migration, run php artisan make:model Student -m, define columns, run php artisan migrate, and use Student::create()/Student::all() after configuring fillable fields and validation. This extension introduces persistence and Eloquent beyond the core five-lesson revision.

Revision questions — short answer

- What is Laravel? State any five features.
- What is Composer, and why does Laravel need it?
- What is the purpose of the .env file?
- What is the role of routes/web.php?
- Explain MVC with one example for each component.
- What is the difference between GET and POST?
- What is a route parameter? Give an example.
- Why are named routes useful?
- What is a Blade view and where is it stored?
- What is the difference between {{ }} and {!! !!}?
- Explain @if, @foreach and @forelse.
- What is template inheritance? Explain @extends, @section and @yield.
- What is a controller? How do you generate one?
- What are the seven standard resource controller methods?
- What is middleware? What does $next($request) do?
- Why does Laravel return 419 for a POST request without a CSRF token?
- How do you inspect all registered routes?
- How are routes, controllers and views connected?

Practical viva / code-reading questions

- Explain this code: Route::get('/students', [StudentController::class, 'index'])->name('students.index').
- What does compact('students') do when returning a view?
- What does route('students.show', ['id' => 5]) generate?
- What does @csrf add to an HTML form?
- What does @method('PUT') do?
- What is the output of $loop->iteration for the first item?
- What happens if middleware returns a response without calling $next($request)?
- What does Route::resource('students', StudentController::class) generate?
- Why should user-provided values normally be displayed with {{ $value }}?
- Why will data stored only in a local PHP array not persist between HTTP requests?

Answer key — concise

- Laravel is a PHP framework that provides routing, MVC conventions, Blade, middleware, Artisan, validation and database tools.
- Composer manages PHP dependencies and autoloading.
- .env contains environment-specific configuration and secrets; keep it private.
- routes/web.php defines browser-facing web routes and uses the web middleware stack.
- Model = data, View = presentation, Controller = request/application coordination.
- GET retrieves; POST submits data or creates a resource.
- A route parameter captures a URI segment, e.g. /users/{id}.
- Named routes avoid hardcoded URLs and make URL generation resilient to URI changes.
- Blade templates are .blade.php files in resources/views.
- {{ }} escapes output; {!! !!} renders raw HTML.
- @if conditionally renders; @foreach iterates; @forelse also handles an empty collection.

- @extends selects a parent layout; @section defines content; @yield displays it.
- Controller groups request-handling actions; generate with make:controller.
- index, create, store, show, edit, update, destroy.
- Middleware filters a request; $next passes it onward.
- 419 commonly indicates invalid/missing CSRF token or session on a web request.
- php artisan route:list.
- Route maps request to controller; controller prepares data/response; view renders the interface.

Quick revision sheet

Remember	Syntax / command
Create project	composer create-project laravel/laravel project-name
Run server	php artisan serve
List routes	php artisan route:list
Create controller	php artisan make:controller StudentController
Create resource controller	php artisan make:controller StudentController --resource
Create middleware	php artisan make:middleware CheckAge
GET route	Route::get('/path', [Controller::class, 'method']);
POST route	Route::post('/path', [Controller::class, 'store']);
Named route	->name('students.index')
Generate URL	route('students.index')
Return view	return view('students.index', compact('students'));
Blade output	{{ $variable }}
CSRF token	@csrf
Method spoofing	@method('PUT')
Layout inheritance	@extends('layouts.app'), @section, @yield
Partial	@include('partials.nav')
Resource routes	Route::resource('students', StudentController::class);
Middleware continuation	return $next($request);